

EXPERIMENT NO.5

Aim: To Study the Deployment of Chain code in Hyperledger fabric.

Theory:

What is a Chain code - A chaincode typically handles business logic agreed to by members of the network, so it may be considered as a “smart contract”. State created by a chaincode is scoped exclusively to that chaincode and can’t be accessed directly by another chaincode. However, within the same network, given the appropriate permission a chaincode may invoke another chaincode to access its state. In simple terms Chaincode is the term used in place of Smart Contract within Hyperledger Fabric.

Chaincode runs in a secured Docker container isolated from the endorsing peer process. Chaincode initializes and manages ledger state through transactions submitted by applications. A chaincode typically handles business logic agreed to by members of the network, so it may be considered as a “smart contract”.

A Chaincode (or “smart contract”) is an application that runs on top of the underlying architecture to enforce business rules and maintain the state. This application can define its own data structures, or use the types of services exposed by other chain codes running in parallel on the same peer node.

- The node that participates in the consensus protocol validates blocks and distributes them to other peers. Each node has access to all information required for this task and runs an internal chaincode that enforces business rules.
- Chaincode is the program that is running on a peer node.
- There are many chain codes out there and they all run the same way, enforcing business rules on transactions.
- Chaincode can be written in any language supported by the underlying platform.

What chaincode looks like

Each chaincode is implemented as an isolated program that maintains its own private state on the ledger. (Note: A chaincode may allow other chaincodes to query its private state.) Chaincodes are essentially arbitrary code, but they conform to a common interface that the Peer uses to control them.

Chaincode Interface

Each chaincode implements an Init method and an Invoke method. The first is used for setup, and the second is used to do the main work of the chaincode. Both methods accept a Chaincode Stub Interface parameter, which carries a client for interacting with the ledger and for querying other chaincodes. The ChaincodeStubInterface also holds a list of arguments passed to the chaincode, which allows the chaincode to implement different behaviours for different scenarios. For example, a single chaincode may implement multiple functions. A client can pass arguments to select which specific chaincode function it wants to invoke — just like passing subcommands to a command-line utility (e.g. the test in go test).

Chaincode Lifecycle

In some scenarios, multiple entities must agree on a chaincode before it can be used. Each entity must be able to review the source code of the chaincode and sign it to prevent tampering. The package format used to pass around the chaincode implementation is called Chaincode Deployment Spec, which includes the source code, the policies for instantiating (setting up and running) the chaincode application, and the list of entities that have agreed on the chaincode.

Once a chaincode has been properly endorsed, it can be installed and instantiated on Peers in the network. During the instantiation process, the Peer uses Docker to run a container with the chaincode inside. The Peer is responsible for managing the chaincode container's lifecycle and networking.

This detail is the source of some contention, as it gets in the way of running Hyperledger Fabric on Kubernetes (jira.hyperledger.org/browse/FAB-7406). Specifically, Kubernetes is designed around Pods, which have their own networking model — different from plain Docker containers. Furthermore, since chaincode containers are managed directly using Docker, they are invisible to Kubernetes. It's likely that a small change to this part of the architecture would help Hyperledger Fabric be compatible with more platforms.

Hyperledger Fabric

Hyperledger Fabric is an open-source distributed blockchain implementation. Hyperledger Fabric is a permission based DLT framework for creating enterprise grade business solution using blockchain technology. Hyperledger is platform for distributed ledger solutions, underpinned by a modular architecture delivering high degrees of confidentiality, resiliency, flexibility and scalability. An important piece of its architecture is chaincode. The core of any blockchain platform is the ledger, a shared, tamper-proof history of the universe. Specifically, it's a record of all transactions within its scope. Chaincode is the term for programs that run on top of the blockchain to implement the business logic of how applications interact with the ledger. When a transaction is proposed, it triggers chaincode that decides what state change should be applied to the ledger. For example, chaincode for the sale of bitcoin would check that the seller actually has bitcoin to sell.

Since chaincode implements logic agreed on by members of the blockchain network, it is sometimes called smart contracts. However, chaincode can be used to implement functionality beyond smart contracts. The only constraint is that all chaincodes are controlled by a Peer according to the same interface — which is flexible enough that Hyperledger Fabric actually uses chaincodes to implement some system-level features.

Types of Chaincodes in Hyperledger Fabric

In Hyperledger Fabric, there are two different types of chaincodes:

1. Orderer Chaincode:

This type of chaincode is executed by an orderer node (a peer with special tagging configured in the network identity). The orderer executes this chaincode for each transaction and subsequently enforces the business logic associated with this transaction by representing it as an "Order" object. The order is then sent to the participants in the peer-to-peer network ("Peer") and they are responsible for processing it (as long as they confirm that the transaction is valid). This type of chaincode is executed by an orderer node (a peer with special tagging configured in the network identity). The orderer executes this chaincode for each transaction and subsequently enforces the business logic

associated with this transaction by representing it as an “Order” object. The order is then sent to the participants in the peer-to-peer network (“Peer”) and they are responsible for processing it (as long as they confirm that the transaction is valid).

2. Core Chaincode:

This type of chaincode is executed by a core node (a peer that runs multiple chaincodes). The core node has special tagging configured in the network identity. The core node executes this chaincode for each transaction but represents it as a new message that is sent using the Messaging service. As mentioned above, chaincode will run on the peer nodes within the Hyperledger Fabric network. When running a chaincode, there are several things that will be enforced on transactions by chaincode. A peer group can be seen as a logical cluster of peers that are connected to each other through a leader. Each peer in this group will be running the chaincode of the same type.

What is System Chaincodes?

- A system chaincode is a set of functions that provides the blockchain runtime with information about the status of a transaction, while still allowing the transaction to execute. It can be used to validate or reject transactions, or to obtain additional information about Blockchain interactions.
- In Hyperledger Fabric, system chaincodes are usually written in a programming language based on JavaScript (e.g., Nodejs) and run in a container as an isolated process. A developer can write custom application-specific chaincodes by using one of many available SDKs for systems programming languages such as C++, Go, Rust and Python.
- Hyperledger Fabric is a blockchain-based distributed ledger with a modular architecture, which is, in general, compatible with Hyperledger Fabric.
- There are several organizations that provide support for the Hyperledger Fabric project, and Hyperledger is one of the key contributors to this effort. It is said to be an open-source distributed ledger framework designed by the Linux Foundation.
- It has been inspired by the Bitcoin Blockchain as well as the Ethereum Platform and its purpose is to have an open source blockchain system in which each participating enterprise can design and build their own custom applications to interact with the blockchain “to improve their operational processes or business model”.

Chaincode For Developers in Blockchain

Chaincode is a program that lives in the blockchain and it executes transactions when instructed. It offers an alternative to using scripts to create smart contracts.

- Chaincode is written in JavaScript which allows developers without a background in crypto economics to understand the codebase and use it in their applications.
- Chaincode for developers has been pioneered by Hyperledger Fabric, an open-source framework from Linux Foundation that implements a blockchain architecture with permission ledgers and multi-party computation (MPC).
- As per the spec, chaincode will be used for “stateful business logic”, which does not require any state change by a consensus algorithm.

- Chaincode is code that is invoked by participating nodes in order to validate transactions that are part of a block's blockchain; it can also invoke other transaction types like event subscription or Coinbase transactions. Chaincodes also execute scriptable data-management operations on the blockchain's state database.
- Chaincode is also referred to as smart contracts due to its ability to interact with the blockchain and perform data manipulation functions.
- Several different types of chaincodes are available from Coda Blockchain's Smart Contracts Library, including oracle, multisignature, timestamping, and Keystore. Coda also provides various Coinbase operations for creating/composing transactions that include transaction value and identifying remarks on the blockchain.
- As a means of simplifying development on the blockchain and facilitating interop with legacy systems, Coda provides a number of features such as the use of native libraries like libevent in order to provide access to APIs like libbitcoin or libledger.

Chaincode For Operators

Chaincode is a fabric-specific script written to perform operations within the framework. Hyperledger Fabric's chaincode enables a user with no knowledge of blockchain technology to build and deploy smart contracts and transactions.

- These scripts can be defined as executable pieces of software that act on input data generated by other code.
- The chaincode creates and runs transactions using consensus across nodes in the decentralized system; this helps to enforce the rules set within entities' business logic while taking into account all contributions from other participants in the network, known as actors.
- This includes both transactions between actors, such as customers or suppliers, but also interactions with fabric-specific functions like transaction ordering or reconciliation.
- Within the context of blockchain development, smart contracts also refer to contract code that is enforced by the distributed blockchain ledger and executed automatically when a condition specified in the contract code is triggered.
- In this context, smart contracts are executable programmable logic (programs) that are stored on the blockchain, which can only be run by authorized parties. For operators, they can be used as a means of upgrading their distributed ledger systems to provide new functionalities, such as allowing users to issue assets and make transactions with them.
- Chaincode provides an interface for smart contracts to be programmed via Chaincode. Coda's Chaincode performs the following functions:
- Chaincode is launched in one of two ways in Coda: Either by invoking it via the CLI or programmatically via SDK calls. In order for the node operator to launch a chaincode, one must first authorize launch permissions by providing a list of node IDs for which the chaincode is authorized to run.
- Chain codes should primarily be written using Go, but other languages such as Python and NodeJS can also be used.

Hyperledger Fabric 2.0 Chaincode deployment steps

Here are the steps in order to succeed with chaincode deployment in Hyperledger Fabric 2.0

1. Install Prerequisites

- Install Docker Desktop
- After successful Installation → Check **docker --version**
- Install WSL 2 → using command **wsl --install**
- After successful installation → Check **wsl --list --verbose**
- Install Go (Golang) → using command **sudo apt install golang -y**
- After successful installation → Check **go version**
- Install node.js & NPM → using command **sudo apt install nodejs npm -y**
- After successful installation → Check **node -v & npm -v**

```
root@LAPTOP-JTRP63CF: ~  
root@LAPTOP-JTRP63CF:~# docker --version  
Docker version 24.0.7, build afdd53b  
root@LAPTOP-JTRP63CF:~# go version  
go version go1.22.2 linux/amd64  
root@LAPTOP-JTRP63CF:~# node -v  
v18.19.1  
root@LAPTOP-JTRP63CF:~# npm -v  
9.2.0
```

```
C:\Users\meets>wsl --list --verbose  
NAME                STATE      VERSION  
* docker-desktop    Stopped    2  
  Ubuntu            Stopped    2  
  docker-desktop-data Stopped    2
```

2. Install Hyperledger Fabric Binaries

- Open Ubuntu (WSL) and run the following commands:
mkdir -p ~/fabric && cd ~/fabric
- Download Hyperledger Fabric binaries and Docker images:
curl -sSL https://bit.ly/2ysbOFE | bash -s
- Verify the installation:
cd fabric-samples/test-network
./network.sh version

```
Generating CCP files for Org1 and Org2  
[+] Running 7/7  
✔ orderer.example.com 6 layers [#####] 0B/0B Pulled 7.6s ✔6414378b6477 Already exists  
✔b67936ae0f59 Pull complete 0.0s 1.0s  
✔4774c04a10bb Pull complete 2.3s  
✔dd98d562807b Pull complete 1.0s  
✔9bb334e1c994 Pull complete 2.1s  
✔6cc91d16b133 Pull complete 2.1s  
WARN[0007] Found orphan containers ([ca_org1 ca_org2 ca_orderer]) for this project. If you removed or renamed this service in your compose file, you can run this command with the --remove-orphans flag to clean it up.  
[+] Running 6/6  
✔Volume "compose_orderer.example.com" Created 0.0s  
✔Volume "compose_peer0.org1.example.com" Created 0.0s  
✔Volume "compose_peer0.org2.example.com" Created 0.0s  
✔Container peer0.org1.example.com Started 1.2s  
✔Container peer0.org2.example.com Started 1.2s  
✔Container orderer.example.com Started 1.2s
```

3. Start the Hyperledger Fabric Network

- Bring up the network and create a channel:
./network.sh up createChannel -ca
- Verify the network is running:
docker ps

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
a8a52eadf170	hyperledger/fabric-orderer:latest	orderer	"orderer"	2 seconds ago	Up Less than a second	0.0.0.0:7050->7050/tcp, 0.0.0.0:7053->7053/tcp, 0.0.0.0:9443->9443/tcp
903f132cfc35	hyperledger/fabric-peer:latest	peer0.org1.example.com	"peer node start"	2 seconds ago	Up Less than a second	0.0.0.0:7051->7051/tcp, 0.0.0.0:9444->9444/tcp
75bbc593c409	hyperledger/fabric-peer:latest	peer0.org2.example.com	"peer node start"	2 seconds ago	Up Less than a second	0.0.0.0:9051->9051/tcp, 7051/tcp, 0.0.0.0:9445->9445/tcp
da91ecc47e0c	hyperledger/fabric-ca:latest	ca_org1	"sh -c 'fabric-ca-se..."	22 seconds ago	Up 19 seconds	0.0.0.0:7054->7054/tcp, 0.0.0.0:17054->17054/tcp
320d40b2aeb8	hyperledger/fabric-ca:latest	ca_org2	"sh -c 'fabric-ca-se..."	22 seconds ago	Up 18 seconds	0.0.0.0:8054->8054/tcp, 7054/tcp, 0.0.0.0:18054->18054/tcp
3760d6adelc7	hyperledger/fabric-ca:latest	ca_orderer	"sh -c 'fabric-ca-se..."	22 seconds ago	Up 18 seconds	0.0.0.0:9054->9054/tcp, 7054/tcp, 0.0.0.0:19054->19054/tcp

4. Deploying Chaincode

- Deploying a sample chaincode to the network using the command in Ubuntu (WSL):
./network.sh deployCC -ccn basic -ccp ../asset-transfer-basic/chaincode-go -ccl go
- Once the command finishes, check if the chaincode is successfully deployed:
docker ps

```
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-go
- CC_SRC_LANGUAGE: go
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-go
- CC_SRC_LANGUAGE: go
- CC_VERSION: 1.0
Vendoring Go dependencies at ../asset-transfer-basic/chaincode-go
```

5. Interacting with the chaincode

- Invoke the chaincode (Create an Asset)

Command:

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile /root/fabric/fabric-samples/test-
network/organizations/ordererOrganizations/example.com/tlsca/tlsca.example.co
m-cert.pem -C mychannel -n basic --peerAddresses localhost:7051 --
tlsRootCertFiles /root/fabric/fabric-samples/test-
network/organizations/peerOrganizations/org1.example.com/tlsca/tlsca.org1.exe
```

```
mple.com-cert.pem --peerAddresses localhost:9051 --tlsRootCertFiles
/root/fabric/fabric-samples/test-
network/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.exa
mple.com-cert.pem -c '{"function":"CreateAsset","Args":["asset1", "blue",
"10", "Tom", "100"]}'
```

Output:

This creates an asset with:

- ID: asset1
- Color: blue
- Size: 10
- Owner: Tom
- Value: 100

```
Query installed successful on peer0.org1 on channel
Using organization 1
+ peer lifecycle chaincode approveformyorg -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile /root/fabric/fabric-samples/tes
t-network/organizations/ordererOrganizations/example.com/tlsca/tlsca.example.com-cert.pem --channelID mychannel --name basic --version 1.0 --package-id basi
c_1.0:b2974d4b301358c998b17154ea19f8376cd812fddfd96150a17888520322630d --sequence 1
+ res=0
2025-02-20 18:07:08.541 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [f4dba709f5e67cffd644fef15f15b3fd1bbc054f22d80da5c9bf76fff9fee52] committed with st
atus (VALID) at localhost:7051
Chaincode definition approved on peer0.org1 on channel 'mychannel'
```

6. Query the Created Asset

- Check if asset exist in the blockchain by querying it
peer chaincode query -C mychannel -n basic -c
'{"Args":["ReadAsset","asset1"]}'

Output:

Json

```
{
  "ID": "asset1",
  "Color": "blue",
  "Size": 10,
  "Owner": "Tom",
  "AppraisedValue": 100
}
```

7. Query the Updated Asset

- Verify if the asset was updated successfully
peer chaincode query -C mychannel -n basic -c
'{"Args":["ReadAsset","asset1"]}'

Output:

Json

```
{
  "ID": "asset1",
  "Color": "blue",
  "Size": 10,
```

```
"Owner": "Alice",
"AppraisedValue": 200
}
```

8. Verify the Asset Deletion

- Try querying **asset1** again:
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset1"]}'

Output:

Javascript

Error: asset1 not found

9. Shutting Down the Fabric Network

- Bring Down the network
cd ~/fabric/fabric-samples/test-network
./network.sh down

Output:

Stopping network

Removing Docker containers...

Removing Docker volumes...

Network shut down successfully

```
root@LAP10P-JTRP63CF:~/fabric/fabric-samples/test-network# cd ~/fabric/fabric-samples/test-network
root@LAP10P-JTRP63CF:~/fabric/fabric-samples/test-network# ./network.sh down
Using docker and docker-compose
Stopping network
[+] Running 14/14
✔Container peer0.org1.example.com      Removed      1.5s
✔Container peer0.org2.example.com      Removed      1.7s
✔Container ca_orderer                  Removed      11.0s
✔Container orderer.example.com         Removed      0.9s
✔Container ca_org1                    Removed      10.8s
✔Container ca_org2                    Removed      10.9s
✔Volume compose_orderer.example.com    Removed      0.0s
✔Volume compose_orderer3.example.com   Removed      0.0s
✔Volume compose_peer0.org1.example.com  Removed      0.1s
✔Volume compose_orderer4.example.com    Removed      0.0s
✔Volume compose_peer0.org2.example.com  Removed      0.0s
✔Volume compose_orderer2.example.com    Removed      0.0s
✔Volume compose_peer0.org3.example.com  Removed      0.1s
✔Network fabric_test                   Removed      0.3s
```

Conclusion: Thus, we have successfully studied and implemented deployment of chaincode in Hyperledger Fabric.